

1. Context

Modern datacenters are highly dynamic environments built on:

- Virtualization (VMs)
- Containers (e.g., Kubernetes-based workloads)
- Software-defined networking (SDN)
- Cloud-native microservices architectures

Services are elastic, distributed, and continuously scaled. However, this dynamism introduces:

- Performance variability
- Resource contention
- Cascading failures
- Limited observability across layers (application ↔ OS ↔ network ↔ hardware)

Traditional monitoring solutions are often:

- Reactive instead of proactive
- Focused on isolated metrics (CPU, RAM)
- Not integrated with control or mitigation mechanisms

There is a growing need for **dynamic, automated observability and control systems** that can:

- Detect anomalies early
- Correlate multi-layer metrics
- Support automated remediation

2. Problem

In dynamic datacenters:

1. Performance degradation is often detected too late.
2. Root-cause analysis is complex due to:
 - a. Multi-layer architectures
 - b. Distributed services
 - c. Ephemeral workloads
3. Monitoring systems generate large volumes of metrics and logs but:

- a. Lack intelligent correlation
 - b. Do not provide predictive insights
4. Fault injection and stress testing are rarely integrated into continuous monitoring frameworks.

Core problem:

How can we design an intelligent, unified tool to collect metrics, stress resources, and benchmark hardware capable of detecting, analyzing, and mitigating performance degradation in dynamic datacenter environments?

3. Goals

General Goal

The primary objective of this project aims to explore the design of an intelligent framework for monitoring and environmental control of modern datacenters. Capable of analyzing system metrics across multiple layers of the infrastructure.

The proposed approach seeks to:

- **Improve observability through autonomous multi-layer metric collection and analysis;**
- **Support automated mechanism for stress testing and fault injection and benchmarking, enabling analysis and mitigating performance degradation;**
- Ease early detection of anomalies, making the system more proactive;
- Apply ML techniques for anomaly detection, performance prediction and bottleneck identification;
- **Provide a dashboard for centralized visualization and interaction.**

4. Methodology

To support a modular project across three independent teams, we adopted an **Iterative, Component-Based Agile methodology**. This allowed us to develop features in parallel while using weekly stakeholder feedback loops to continuously refine our direction.

Initially we divided ourselves in 3 teams, one for the collector that started with 3 elements of our group, another element worked on the UI/Frontend and one for the Tests and Fault Injection.

5. Methods and Solutions

Divided in 3 teams: Collector, Stress Test, UI

Falar sobre a ligação inicial do watch.sh do collector que verificava se chegava algum .json com o pedido do user e colocava num folder o .csv com os dados

6. Demo

7. Future work

- Dataset formation: Performance definition, anomaly/failure definition;
- Apply ML techniques for anomaly detection, performance prediction and bottleneck identification;
- Build security and a role management system for the dashboard (RBAC), user information database;
- Add other benchmark methods such as SPEC;
- Improving dashboard graphic capabilities.
- Extension of data collection for containers/microservices, GPU, Network attached devices, latencies (network and memory), Storage (allocation and NFS)
- If not done until friday: Ansible automation for new performance testing nodes, stress test concurrency, schedule collection.

